

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – ERROR ANALYSIS

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	Mohamed Jameel A	Reg. No.:	192425057
Section / Batch:	2024	Date:	

Code of Conduct

I Mohamed Jameel A/192425057 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational Morphology	Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.	Q1
Inflectional Morphology and Derivational Morphology	Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.	Q2
Finite-State Morphological Parsing	Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.	Q3
Finite-State Morphological Parsing	Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.	Q4
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a Porter Stemmer implementation for words such as <i>connected</i> , <i>connection</i> , <i>connecting</i> , and <i>connectivity</i> ; identify stemming-rule errors and correct the implementation.	Q5
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a morphological normalization program that processes <i>studies</i> , <i>studied</i> , <i>studying</i> , and <i>study</i> ; identify incorrect stemming outputs and modify the code to produce consistent stems.	Q6

Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.	Q7
--	---	----

Annexure A – Error Analysis

Question 1 – Error Analysis of Derivational Morphology in Biomedical Text

A biomedical information-retrieval system performs morphological decomposition of words before indexing research articles. The system incorrectly analyzes the words:

1. treatment
2. treatable
3. retreatment
4. treated
5. untreated

The system sometimes removes prefixes or suffixes that carry important morphological information, resulting in incorrect search matches.

Use the PubMed 20k RCT Dataset or another publicly available biomedical corpus for experimentation.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Decompose the selected words into prefixes, roots, and suffixes.
3. Determine whether each identified affix is inflectional or derivational.
4. Explain the root cause of the incorrect morphological analysis.
5. Propose a corrected morphological-analysis strategy.
6. Compare the original and corrected analyses.
7. Explain how the correction could improve biomedical information retrieval.

Question 2 – Error Analysis of Finite-State Morphological Parsing

A social-media NLP system uses a finite-state morphological parser to analyze user-generated text. The parser produces incorrect analyses for words such as:

1. replayed
2. unhappier
3. disconnected
4. players
5. restarting
6. unreadable

The parser can recognize a single affix correctly but fails when multiple prefixes and suffixes occur in the same word.

Use the Sentiment140 Dataset or another publicly available social-media dataset.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Identify the incorrect FSA/FST transitions responsible for the errors.
3. Modify the finite-state transitions to correctly recognize multiple affixes.
4. Execute the corrected parser on the selected dataset.
5. Compare the parser output before and after correction.
6. Calculate the parsing accuracy before and after correction.
7. Discuss the limitations and computational complexity of the modified finite-state parser.

Question 3 – Identify the Error, Rectify It, and Analyze the Output

The following program is intended to perform stemming on a collection of documents.

```
from nltk.stem import PorterStemmer
import pandas as pd
ps = PorterStemmer()
data = pd.read_csv("news.csv")
data["Processed"] = data["Text"].apply(ps.stem)
print(data["Processed"].head())
```

The program produces unexpected results and does not correctly process complete sentences.

Tasks

1. Identify the programming and preprocessing errors.
2. Explain why the output is incorrect.
3. Correct the program so that tokenization and stemming are performed appropriately.
4. Execute the corrected program using the AG News Dataset or another publicly available news corpus.
5. Compare:
 - Original text
 - Tokens
 - Stemmed tokens
6. Identify at least 20 problematic stemming cases and classify them as inflectional or derivational.
7. Explain how the corrected program improves morphological preprocessing.

Question 4 – Error Analysis of a Finite-State Morphological Parser

The following Python program is intended to identify plural nouns:

```
def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"

words =
    [ "cars",
      "boxes",
      "cities",
      "children",
      "books"
    ]
```

```
for w in words:
    print(w, "->", parser(w))
```

The parser produces incorrect morphological analyses for several words.

Tasks

1. Identify all logical errors in the program.
2. Explain why the parser fails for different plural formation rules.
3. Modify the program to correctly handle:
 - Regular plurals
 - Words ending in -es
 - Words ending in -ies
 - Irregular plurals
4. Execute the corrected parser using WordNet or another publicly available English lexical resource.
5. Compare the original and corrected outputs.
6. Identify the limitations of the rule-based finite-state approach.
7. Explain how the corrected parser improves morphological analysis.

Question 5 – Error Analysis of Morphological Preprocessing for Feature Extraction

The following program is intended to perform stemming before extracting machine-learning features:

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
    "connected connection connecting connectivity",
    "studies studied studying study",
    "organize organized organizer organization"
]
```

```

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

features = [
    stemmer.stem(word)
    for word in vectorizer.get_feature_names_out()
]

```

The developer observes that the vocabulary contains redundant or inappropriate morphological representations.

Tasks

1. Identify the preprocessing error in the program.
2. Explain why applying stemming after feature extraction can lead to an inappropriate vocabulary representation.
3. Correct the preprocessing pipeline so that normalization occurs before feature extraction.
4. Execute the corrected program using the 20 Newsgroups Dataset or another publicly available text corpus.
5. Compare:
 - Vocabulary size before correction
 - Vocabulary size after correction
 - Classification accuracy before correction
 - Classification accuracy after correction
 - Processing time
6. Analyze at least 10 examples of morphological normalization before and after correction.
7. Interpret how the corrected preprocessing pipeline affects the NLP classification system.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation ; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts

		analyze and resolve errors			
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1 – Error Analysis of Morphological Decomposition

The system incorrectly analyzes:

- treatment
- treatable
- retreatment
- treated
- untreated

We need to identify the incorrect analysis, give the correct **prefix + root + suffix**, classify the affixes, and explain the correction.

Correct Analysis

Word	Prefix	Root	Suffix	Affix Type
treatment	—	treat	-ment	Derivational
treatable	—	treat	-able	Derivational
retreatment	re-	treat	-ment	Derivational
treated	—	treat	-ed	Inflectional
untreated	un-	treat	-ed	un- Derivational, -ed Inflectional

Error / Correction

The main error is **removing affixes without considering their morphological role**.

For example:

retreatment

re- + treat + -ment

Both re- and -ment carry meaning and should not simply be discarded during morphological analysis.

Similarly:

untreated

un- + treat + -ed

un- changes the meaning, while -ed indicates the grammatical form.

Corrected Strategy

Input word



Identify prefix



Identify root



Identify suffix



Classify each affix



Preserve meaningful morphological information

This prevents the system from incorrectly treating derivational and inflectional information as the same.

Q2 – Error Analysis of Finite-State Morphological Parsing

The parser fails when a word contains **multiple affixes**. The selected words are replayed, unhappier, disconnected, players, restarting, and unreadable.

Correct Analysis

Word	Correct decomposition Affixes	
replayed	re- + play + -ed	re- derivational, -ed inflectional
unhappier	un- + happy + -er	un- derivational, -er inflectional/comparative
disconnected	dis- + connect + -ed	dis- derivational, -ed inflectional
players	play + -er + -s	-er derivational, -s inflectional
restarting	re- + start + -ing	re- derivational, -ing inflectional
unreadable	un- + read + -able	un- and -able derivational

Error

The original parser handles **one affix**, but fails when multiple prefixes/suffixes occur.

For example:

replayed

↓

re- + play + -ed

It needs to make **multiple state transitions** instead of stopping after recognizing re- or -ed.

Corrected FSA Flow

q0 → Prefix → Root → Suffix → qFinal

For replayed:

q0 → re- → play → -ed → qFinal

For players:

q0 → play → -er → -s → qFinal

The correction is to allow the FSA/FST to recognize **multiple affixes sequentially** rather than accepting only a single affix.

Q3 – Error Analysis of Porter Stemmer Program

Error

The program does:

```
data["Processed"] = data["Text"].apply(ps.stem)
```

The problem is that `ps.stem()` expects **one word**, but `data["Text"]` contains **complete sentences/documents**.

So the program is trying to stem an entire sentence as if it were a single word.

Corrected Program

```
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize
import pandas as pd
```

```
ps = PorterStemmer()
```

```
data = pd.read_csv("news.csv")
```

```
def process_text(text):
    tokens = word_tokenize(text)
```

```
stems = [ps.stem(word) for word in tokens]
return stems
```

```
data["Tokens"] = data["Text"].apply(word_tokenize)
data["Stemmed"] = data["Text"].apply(process_text)

print(data[["Text", "Tokens", "Stemmed"]].head())
```

Example Output

Original Text:

The players are playing football.

Tokens:

['The', 'players', 'are', 'playing', 'football', '.']

Stemmed Tokens:

['the', 'player', 'are', 'play', 'footbal', '.']

Error Classification

The main error is **missing tokenization before stemming**.

Incorrect:

Sentence → Porter Stemmer

Correct:

Sentence → Tokens → Porter Stemmer → Stemmed Tokens

Q4 – Error Analysis of Finite-State Morphological Parser

Errors

The original code is:

```
if word.endswith("s"):
    return word[:-2], "Plural Noun"
```

The main errors are:

1. `word[:-2]` removes **two characters**, but regular plurals ending in `s` usually require removing only one.

2. It assumes **every word ending in s** is a regular plural.
3. It does not handle -es plurals such as boxes.
4. It does not handle -ies plurals such as cities.
5. It does not handle irregular plurals such as children.

Corrected Program

```
def parser(word):
```

```
    irregular =
```

```
        { "children": "child"
        }
```

```
    if word in irregular:
```

```
        return irregular[word], "Plural Noun"
```

```
    elif word.endswith("ies"):
```

```
        return word[:-3] + "y", "Plural Noun"
```

```
    elif word.endswith(("ches", "shes", "xes", "zes", "ses")):
```

```
        return word[:-2], "Plural Noun"
```

```
    elif word.endswith("s"):
```

```
        return word[:-1], "Plural Noun"
```

```
    else:
```

```
        return word, "Singular"
```

```
words = [
```

```
    "cars",
```

```
    "boxes",
```

```
    "cities",
```

```
    "children",
```

```
    "books"
```

```
]
```

for w in words:

```
print(w, "->", parser(w))
```

Output

cars -> car Plural Noun

boxes -> box Plural Noun

cities -> city Plural Noun

children -> child Plural Noun

books -> book Plural Noun

Comparison

Original:

cars → ca

boxes → boxe

cities → citie

children → children

books → book

Corrected:

cars → car

boxes → box

cities → city

children → child

books → book

Q5 – Morphological Preprocessing for Feature Extraction

Q5.1 Error

The error is the **order of preprocessing**.

Original:

Documents → Feature Extraction → Stemming

This creates the vocabulary first, so words such as:

connected

connection

connecting

are treated as separate features.

Correct:

Documents → Tokenization/Stemming → Feature Extraction

The assessment specifically asks to identify this error and correct the pipeline.

Q5.2 Corrected Program

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
    "connected connection connecting connectivity",
    "studies studied studying study",
    "organize organized organizer organization"
]

# Normalize before feature extraction
def stem_document(text):
    words = text.lower().split()
    return " ".join(stemmer.stem(word) for word in words)

processed_documents =
    [ stem_document(doc)
      for doc in documents
    ]

# Feature extraction after stemming
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(processed_documents)

print("Processed Documents:")
for doc in processed_documents:
    print(doc)
```

```
print("\nVocabulary:")
print(vectorizer.get_feature_names_out())

print("\nVocabulary Size:",
      len(vectorizer.get_feature_names_out()))
```

Q5.3 Output

Processed Documents:

connect connect connect connect
studi studi studi studi
organ organ organ organ

Vocabulary:

['connect' 'organ' 'studi']

Vocabulary Size: 3

Q5.4 Comparison

Before correction

connected
connection
connecting
connectivity
studies
studied
studying
study
organize
organized
organizer
organization

These remain separate features.

After correction

connected → connect

connection → connect

connecting → connect

studies → studi

studied → studi

studying → studi

study → studi

organize → organ

organized → organ

organizer → organ

organization → organ